

picam_detector.py

Line-by-Line Explanation

BirdGuard — RF-DETR Nano Detector with Pi→Nano UART Laser Trigger

Overview

This script runs on the Raspberry Pi. It continuously captures frames from the Pi camera, runs an RF-DETR Nano object detector on each frame, and when the configured target class (currently "person", swapped to "bird" for production) is detected above a confidence threshold, it sends a single-byte command over UART to the Arduino Nano to switch a laser ON. If the target is absent for a configurable delay, it sends the OFF command. The whole session is recorded to a local .avi file, with annotated JPEG snapshots saved for every frame that had a detection, and each detection is also broadcast as a JSON payload over UDP for any listening dashboard/consumer.

Imports (lines 1–10)

#	Code	Explanation
1	<code>import time</code>	Used for <code>sleep()</code> delays during hardware warm-up and for timestamping the laser cooldown window.
2	<code>import json</code>	Serializes each detection into a JSON payload before sending it over UDP.
3	<code>import serial</code>	PySerial — opens the UART connection to the Arduino Nano that drives the laser.
4	<code>import socket</code>	Standard library UDP socket used to broadcast detection events.
5	<code>import os</code>	Used to create the session/detections directories on disk.
6	<code>import cv2</code>	OpenCV — color conversion, drawing bounding boxes, and writing the output video file.
7	<code>from datetime import datetime</code>	Builds human-readable session folder names and log timestamps.
8	<code>from picamera2 import Picamera2</code>	Official Raspberry Pi camera stack; captures frames as NumPy arrays.
9	<code>from rfdetr import RFDETRNano</code>	The detection model class. RFDETRNano is the smallest/fastest RF-DETR variant, replacing the previous Ultralytics YOLOv8n model.
10	<code>from rfdetr.assets.coco_classes import COCO_CLASSES</code>	A dict mapping COCO numeric class IDs (e.g. 1) to human-readable labels (e.g. "person"). RF-DETR's output only gives numeric class IDs, so this replaces the <code>.names</code> lookup that Ultralytics YOLO provided automatically.

Configuration constants (lines 12–22)

#	Code	Explanation
1 3	<code>SERIAL_PORT = '/dev/serial0'</code>	The Pi's hardware UART device connected to the Nano.
1 4	<code>SERIAL_BAUD = 9600</code>	Baud rate that must match the Nano's Serial.begin() call in its firmware.
1 5	<code>UDP_IP = "127.0.0.1"</code>	Loopback address — the UDP listener runs on the same Pi.
1 6	<code>UDP_PORT = 5005</code>	Port the detection-event listener/dashboard listens on.
1 7	<code>TARGET_CLASS = "person"</code>	Which COCO label triggers the laser. Left as "person" for bench testing; the comment flags it should become "bird" for the real deployment.
1 8	<code>CONFIDENCE_THRESHOLD = 0.5</code>	Minimum detection confidence (0–1) required to count as a hit. Passed straight into RF-DETR's predict() call, so filtering now happens inside the model rather than after the fact.
1 9	<code>LASER_OFF_DELAY = 3.0</code>	Seconds the laser stays on after the last confirmed detection, so it doesn't flicker on/off between consecutive frames that briefly miss the target.
2 0	<code>FRAME_WIDTH = 640</code>	Capture width in pixels.
2 1	<code>FRAME_HEIGHT = 480</code>	Capture height in pixels.
2 2	<code>VIDEO_FPS = 30</code>	The frame-rate constant. Raised from 20 to 30. This single value is reused in two unrelated places later in the script — see the dedicated FPS section below for why that matters.

Session folder setup (lines 24–29)

#	Code	Explanation
2 5	<code>session_name = datetime.now().strftime(...)</code>	Builds a unique folder name like session_2026-07-04_14-30-00 so repeated runs don't overwrite each other.
2 6	<code>SESSION_DIR = f"/home/pi/birdguard-test/{session_name}"</code>	Root folder for this run's output, hardcoded to the Pi's home directory.
2 7	<code>DETECTIONS_DIR = f"{SESSION_DIR}/detections"</code>	Sub-folder where annotated detection frames are saved.
2 8	<code>VIDEO_PATH = f"{SESSION_DIR}/recording.avi"</code>	Path of the full-session video recording.
2 9	<code>os.makedirs(DETECTIONS_DIR, exist_ok=True)</code>	Creates both SESSION_DIR and DETECTIONS_DIR in one call (makedirs creates intermediate directories).

Hardware and model initialization (lines 37–62)

#	Code	Explanation
3 8	<pre>ser = serial.Serial(SERIAL_PORT, SERIAL_BAUD, timeout=1)</pre>	Opens the UART link to the Nano. Wrapped in try/except so the script still runs (camera + recording only, no laser) if no Nano is physically connected — useful for bench testing the vision pipeline alone.
4 7	<pre>sock = socket.socket(socke t.AF_INET, socket.SOCK_DGRAM)</pre>	Creates the UDP socket used to fire detection events out to any listening process.
5 1	<pre>model = RFDETRNano()</pre>	Instantiates the RF-DETR Nano model. On first run this downloads the pretrained COCO weights; on later runs it loads them from the local cache.
- 2	<pre>config = picam2.create_vide o_configuration(main={... }, controls={"FrameRate": VIDEO_FPS})</pre>	First use of VIDEO_FPS. The controls dict is passed straight to libcamera and tells the camera sensor itself to target 30 captured frames per second. This is a request to the hardware/driver, not a guarantee — the sensor mode chosen for 640x480 must support 30fps for this to actually hold.
6 1	<pre>picam2.start()</pre>	Starts the camera stream using the configuration above.

Video writer setup (lines 64–67)

#	Code	Explanation
6 5	<pre>fourcc = cv2.VideoWriter_f ourcc(*'XVID')</pre>	Selects the XVID codec for the output .avi container.
6 6	<pre>writer = cv2.VideoWriter(V IDEO_PATH, fourcc, VIDEO_FPS, (FRAME_WIDTH, FRAME_HEIGHT))</pre>	Second use of VIDEO_FPS. This does NOT control how fast frames are captured or written — it only stamps the metadata in the .avi file header that tells any video player "play these frames back at 30 per second". See the callout below for why this distinction matters.

How the 30fps value is actually used — two independent, unsynchronized meanings: VIDEO_FPS = 30 is read in two places that do very different things. (1) Passed to Picamera2 as controls={"FrameRate": 30}, it asks the camera sensor to capture at 30fps — this affects how often picam2.capture_array() actually hands back a new frame. (2) Passed to cv2.VideoWriter as the fps argument, it only sets the playback speed written into the .avi file's header — it does not throttle or pace the writer.write() calls in any way. The main loop (lines 77–134) is not rate-limited to 30fps by either setting: it runs a full capture → RF-DETR inference → draw → write cycle back-to-back, as fast as each step completes. RF-DETR Nano is a heavier, transformer-based model than the previous YOLOv8n, so on a Raspberry Pi the achievable inference rate may well be below 30fps. If that happens, frames are still captured and written at whatever slower real rate the loop achieves, but the output file is labeled as 30fps — so the saved recording.avi would appear to play back faster than the real event actually happened (motion looks sped up). This is the same real-vs-declared-fps caveat that the companion live_preview.py script calls out explicitly in its own VIDEO_FPS comment. Before relying on VIDEO_FPS = 30 for an accurate recording, the actual sustained loop rate should be measured on the target Pi hardware.

Main detection loop (lines 69–134)

#	Code	Explanation
6 9 - 7 2	<pre>laser_on = False / last_detection_time = 0 / frame_count = 0 / detection_count = 0</pre>	State tracked across loop iterations: whether the laser is currently on, when the target was last seen, and running counters for the end-of-session summary.
7 8	<pre>frame = picam2.capture_array()</pre>	Grabs the next available frame from the camera as a NumPy array in RGB888 layout (per the earlier <code>create_video_configuration</code> format setting).
7 9	<pre>bgr = cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)</pre>	OpenCV drawing/encoding functions expect BGR channel order, so a BGR copy is made for display/recording purposes only.
8 1	<pre>results = model.predict(frame, thresh hold=CONFIDENCE_THRESHOLD)</pre>	Runs RF-DETR inference on the original RGB frame (RF-DETR expects RGB, not BGR — this is why the raw frame is passed here rather than <code>bgr</code>). Returns a Supervision Detections object already filtered by confidence threshold.
8 5 - 8 6	<pre>for xxyy, conf, class_id in zip(results.xxyy, results.confidence, results.class_id): label = COCO_CLASSES.get(int(class _id), str(class_id))</pre>	Iterates every surviving detection in parallel across its three attribute arrays, and resolves the numeric <code>class_id</code> to a readable label via the <code>COCO_CLASSES</code> dict imported at the top of the file.
8 8 - 9 6	<pre>if label == TARGET_CLASS: target_found = True; last_detection_time = time.time(); ...</pre>	For a matching label, records the detection box, marks the target as currently found, resets the "last seen" clock (used by the laser-off delay), and logs it to the console.
9 8 - 1 0 4	<pre>payload = {...}; sock.send to(json.dumps(payload).enc ode(), (UDP_IP, UDP_PORT))</pre>	Packages timestamp, label, confidence, and bounding box into JSON and fires it over UDP for any external consumer/dashboard.
1 0 7 - 1 1 1	<pre>if target_found and not laser_on: ser.write(b'O'); laser_on = True</pre>	On the rising edge of detection (target newly present), sends the single-byte 'O' command to the Nano to turn the laser on.
1 1 3 - 1 1 8	<pre>elif not target_found and laser_on: if time.time() - last_detection_time > LASER_OFF_DELAY: ser.write(b'F'); laser_on = False</pre>	Only turns the laser off once the target has been absent for longer than <code>LASER_OFF_DELAY</code> seconds, preventing rapid on/off flicker from frame-to-frame misses.

#	Code	Explanation
1 2 2 - 1 3 0	<pre>video_frame = bgr.copy(); ... cv2.rectangle(...); cv2.imwrite(...)</pre>	Draws a green bounding box and label/confidence text over every detection on a copy of the BGR frame, and saves that annotated frame as a JPEG snapshot into DETECTIONS_DIR whenever at least one detection occurred this iteration.
1 3 3	<pre>writer.write(video_frame)</pre>	Every frame (detection or not) is appended to the session .avi recording, annotated if applicable.

Shutdown and cleanup (lines 136–151)

#	Code	Explanation
1 3 6 - 1 3 8	<pre>except KeyboardInterrupt: print(...)</pre>	Ctrl+C is the intended way to stop the script; it prints a summary of frames processed and detections logged.
1 4 0 - 1 4 2	<pre>if laser_on and ser: ser.write(b'F')</pre>	Safety net — guarantees the laser is switched off on exit even if the script is interrupted mid-detection.
1 4 3 - 1 4 7	<pre>writer.release(); picam2.stop(); ser.close(); sock.close()</pre>	Releases the video file, stops the camera stream, and closes the serial and UDP sockets cleanly.
1 5 0 - 1 5 1	<pre>if __name__ == "__main__": main()</pre>	Standard Python entry-point guard.